Tarsus: Latency-Optimal Policy-Decoupled Distributed Storage

Nick Tagliamonte, Justin Y. Shi, and Garrett Stoll

Temple University, Philadelphia, Pennsylvania, USA

Abstract. Content addressing authenticates immutable bytes, whereas applications need stable files and directories whose contents, membership, readers, placement, replication, retention, and deletion state change. We present Tarsus, a distributed storage prototype in which an owner-scoped path maps to one DHT key for a signed current generation and its per-object policy. Immutable encrypted chunks, provider claims, current-name search, and tuple coordination remain separate planes, so versions and replicas do not multiply the search namespace. Strict publication exposes a new head only after every block meets its provider policy.

A fixed seven-validator committee can certify a head with five persisted votes. Quorum intersection and durable locks prevent conflicting generations from both committing with at most two Byzantine validators; certified names cannot silently downgrade to the compatibility API. Tests reject insufficient or altered certificates, locked forks, unauthorized state, and tampered data.

Campaigns reach two million current names and 6.4 million retained records while exact lookup remains independent of retained versions over the measured range. In three fresh 20-node deployments, updating a Tarsus directory child rewrites no ancestors and takes 24.43 ms at 256 leaves versus Bee's 1.56 s manifest-plus-feed path. For 64-MiB high-entropy objects, private strict-seven publication takes 4.92 s versus Bee's 80.15 s encrypted upload; adding five-of-seven certification takes 8.93 s versus Bee level-4 erasure's 192.51 s. These results demonstrate the payoff of decoupling the mutable name, immutable data, and policy machinery even on the strongest measured path.

Keywords: distributed storage · mutable names · content addressing · replication policy · encrypted storage · Byzantine fault tolerance · DHT.

1 Introduction

An immutable content key answers a narrow and valuable question: "which bytes?" A client can fetch those bytes from an untrusted provider and reject any response whose digest does not match. Applications, however, normally refer to stable logical objects: a project path, model checkpoint, directory, or queue attachment. The bytes at that name change. So can the authorized writers and readers, required replica count, desired placement, retention window, search visibility, and deletion state.

Treating each new content digest as a new logical object pushes these changes into application-specific databases. Treating every version and replica as a search item has the opposite problem: secondary-index size and update work grow with version history and placement. Embedding replication and access rules in the block store couples immutable

storage mechanics to mutable policy. Finally, a local "lock on the content hash" does not serialize changes to a mutable object: the next version necessarily has another hash.

Tarsus addresses this gap with a globally addressable mutable namespace over immutable, encrypted, content-addressed data. Its central principle is *separation by authority and lifetime*. One signed record is the authoritative current head of a logical name. Manifests and chunks are immutable. Provider claims say who currently offers those immutable blocks. A secondary search index contains current public names only. Transient offers, repair work, and application queues use the existing tuple space. A controller reads object policy from the name record and drives automated placement, replication, repair, and collection. Tarsus unifies *automated replication, mutable logical contents, and decoupled control* into one per-object lifecycle: the name remains stable while its bytes and governing policy evolve independently.

Tarsus builds on established content-addressed storage mechanisms. IPFS content is immutable and IPNS publishes signed, versioned mutable pointers [2, 3]. IPFS Cluster supports per-pin replication minimums, maximums, allocation, and repair [5]. Swarm provides signed feeds over immutable content, immutable manifests, encrypted references, and an Access Control Trie (ACT) [7, 8, **?**]. Tarsus advances this design space by making the signed mutable head the authority for per-object policy and by separating exact resolution, search, providers, coordination, and immutable bytes into independently scalable and independently verifiable planes.

This paper contributes:

- directly resolvable signed names for files and directories, with generation CAS, capabilities, leases, tombstones, and current-name search;
- per-object placement, replication, encryption, retention, repair, and strict-publication policy over immutable manifests and chunks;
- five-of-seven authenticated head agreement with durable locks, certified CAS, follower write-back, and downgrade prevention; and
- adversarial, scaling, encrypted-update, repair, and three-deployment 20-node evidence, including matched directory and policy costs against Bee.

2 Model and Requirements

2.1 Objects and principals

A namespace owner holds an Ed25519 key pair and creates a random 256-bit *NamespaceID*. A canonical object name is `tarsus://<namespace-id>/path`. Paths are absolute UTF-8 strings, normalized by removing repeated separators and dot components; parent traversal, NUL, and non-root trailing separators are rejected. The current prototype limits normalized paths to 4096 bytes.

The derived identifier is SHA-256 over a domain tag, NamespaceID, and the normalized path. Owner-signed compatibility heads use `/names/<NameID>`; quorum-certified heads use `/certified-names/<NameID>`. Exact resolution therefore needs neither a secondary index nor a scan across versions or providers. Domain prefixes prevent identifiers and signatures from being reused across protocol roles.

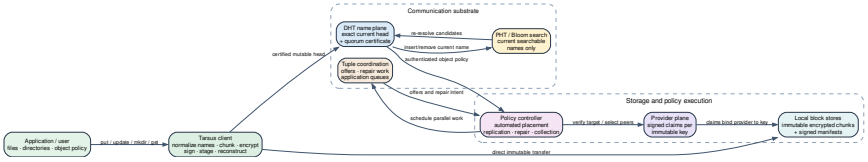


Fig. 1. Policy-decoupled planes. Arrows denote control or verification, not co-location. Search candidates are always re-resolved through the name plane.

Owners may issue time-bounded, path-prefixed capabilities to delegate write, delete, or administrative permission. Storage peers and networks are not trusted for integrity. Reader endpoints and retained private keys are trusted for confidentiality.

2.2 Required semantics

For the compatibility path, two updates submitted from the same expected generation to one fenced exact-name authority must not both commit. For the certified path, no two conflicting records for one name and generation may both obtain valid commit certificates while at most f of $3f + 1$ validators are Byzantine. Every resolved value must be structurally valid, correctly keyed, and signed. Under strict policy, a new head remains invisible until each immutable dependency has the required verified provider claims. Search results are candidates until their current heads are resolved and verified.

Availability and consistency are deliberately separated. Fenced owner handoff may delay or reject writes. The system does not promise linearizable write availability when the authority is partitioned. This is a different claim from immutable-data availability: previously published blocks can remain retrievable while a head update is unavailable.

3 Architecture

Figure 1 shows five state planes. Name/policy records are the only mutable authority. Search contains one current public entry per logical name. Provider claims bind storage identities to immutable keys. The data plane contains signed manifests and encrypted chunks. The tuple plane contains transient offers, repair intent, and queues. Splitting these planes prevents replica count or version history from multiplying the searchable namespace and keeps the local block store policy-free. This decomposition drives both central wins: exact-head lookup avoids history scans, and a file update avoids rewriting its directory ancestors.

3.1 Signed mutable heads

A NameRecord contains the format version, NamespaceID, normalized path, derived NameID, object kind, generation, predecessor-record hash, current manifest key or

tombstone, owner and optional delegation, policy, signer, timestamp, nonce, and Ed25519 signature. Directory records additionally name child NameID values. Records use deterministic DAG-CBOR; validators reject an alternative encoding before signature verification. RFC 8949 describes deterministic CBOR requirements [16].

Generation zero is owner-signed and has no predecessor. Each update increments the generation by exactly one and hashes the complete preceding signed record. Immutable identity fields cannot change. A delegate must present an unexpired owner-signed capability whose path prefix and permission cover the mutation. Policy changes require administrative permission and tombstones require delete permission. Independent signing domains cover name records, capabilities, manifests, provider claims, and leases, preventing a signature created for one role from authorizing another.

The client submits the signed record with `expected_generation`. The deterministic exact-name owner verifies the current generation, predecessor, signature, and authorization, then performs one CAS. An equal-generation fork is rejected rather than broken using arrival time or lexicographic bytes. A valid lease reduces duplicated staging work, but CAS remains mandatory.

3.2 Directories, rename, and leases

A live directory is itself a signed mutable name. Instead of a dummy content manifest, its canonical record contains a sorted, duplicate-free set of at most 4096 child NameID values. Before create or update, each child is resolved and must be live, have the same owner and namespace, and have the directory as its immediate normalized parent; self-membership is rejected. Consequently a forged child identifier, cross-namespace splice, stale tombstone, or link to a grandchild cannot enter an honest directory head.

Directories reference child names rather than child content keys. Updating a file changes its head and manifest without rewriting its ancestors, and listing resolves the children concurrently through a bounded 16-worker pool so it returns their current generations. Directory membership itself is copy-on-write: link and unlink create a new signed directory generation and use the same CAS or QC rules as files. Owner-signed path-prefix capabilities then grant write, delete, administration, directory, or lease operations below a concrete subtree without granting them elsewhere in the namespace.

Rename acquires both old and new name leases in ascending NameID order, creates the new name, and tombstones the old one. Sorted acquisition avoids opposite-order deadlock. During an arbitrary partition, recovery uses the two signed heads to complete or reconcile an interrupted rename.

Lease records cover an exact NameID or a normalized namespace subtree. They bind scope, owner, holder, fencing number, issue and expiry time, capability, nonce, and signature. The prototype caps a lease lifetime at five minutes. Acquire, renew, and release update a namespace-scoped CAS registry. It stores the normalized path for exact leases, so an exact name conflicts with every covering subtree and overlapping subtrees conflict even when requests enter different API nodes. Expired entries are pruned; release does not reset the per-scope fencing number. Clients stage and replicate immutable data before taking the short head-update lease. A lease never replaces head CAS and does not supply byte-range locking, which remains an application concern.

4 Immutable Encrypted Data and Policy

4.1 Chunk and manifest format

Objects up to and including 4 MiB use one data block. Larger inputs are read through a bounded 4-MiB prefix and then fixed 1-MiB chunks; the whole logical object need not be buffered before block emission. A signed deterministic manifest records ordered ciphertext keys, integrity tags, lengths, logical size, encryption mode, key epoch, signer, and reader envelopes. The immutable key authenticates each stored ciphertext block; the manifest signature authenticates its ordering and metadata.

For private objects, each epoch receives a random 256-bit data-encryption key. Chunks use XChaCha20-Poly1305 with independently generated extended nonces. Each reader envelope uses an ephemeral X25519 exchange, HKDF-SHA256, and XChaCha20-Poly1305 to wrap the data key. Private per-chunk and whole-object integrity fields are domain-separated HMAC-SHA-256 tags under the epoch key, not raw plaintext hashes. Thus an observer cannot confirm a guess for low-entropy private content from the manifest alone. Reconstruction unwraps the key, verifies every AEAD tag, and verifies the keyed whole-object tag. Ed25519 and X25519 follow RFCs 8032 and 7748 [17, 18].

Within one key epoch, the client maps keyed chunk tags to prior ciphertext references. An unchanged chunk can therefore be reused without encryption or transfer while its equality remains visible only to readers holding the epoch key. Any reader-membership change requires a new epoch, data key, and re-encryption of the current object. Revocation affects future versions; it cannot retract plaintext or keys already obtained by a removed reader.

Public objects retain signed manifests and content hashes but omit encryption nonces and reader envelopes. Searchable object names are public metadata. Setting `searchable=false` removes the secondary entry; it does not make a disclosed NameID or path secret.

4.2 Per-object placement and strict publication

The default policy requests seven total copies, divided into three near-, two middle-, and two far-RTT classes. These are latency-diversity hints only; they do not prove geographic, jurisdictional, rack, or administrative independence. Policies are validated so placement counts sum to the replica target.

After storing a block, a provider returns an Ed25519-signed claim binding its identity, the immutable content key, advertised address, expiry, timestamp, and nonce. The coordinator validates and publishes the claim. A short-lived, revalidated read-through cache is unioned with the DHT view so a propagation prefix cannot make a just-confirmed claim disappear; unsigned routing tokens never count toward the strict predicate.

The mutable head advances only after the manifest and every referenced chunk satisfy its policy. Transfers are parallel, but commit remains fail-closed. If one block reaches only six of seven required providers, the CAS is not attempted and readers continue to resolve the previous generation. This couples publication to availability intentionally while leaving immutable transfer mechanics outside the name store.

4.3 Repair, retention, and deletion

The policy controller periodically verifies live claims for blocks referenced by committed heads. On a missing or unreachable provider, it selects a new candidate using the object's replica and RTT-class policy, transfers and verifies the block, and publishes the replacement claim.

The default retains the latest three generations with a 24-hour collection grace. Deletion first publishes an authorized signed tombstone, removing the name from normal resolution and search, then schedules eligible immutable blocks for best-effort collection. Three meanings must not be conflated: (1) namespace deletion removes the current Tarsus reference; (2) destroying all remaining envelopes and keys can make retained ciphertext unusable to a party that never obtained the key; and (3) the system cannot force a party to erase bytes, keys, or plaintext it already copied.

5 Authenticated Byzantine Profile

Replica count alone never yields Byzantine agreement. Tarsus instead follows the authenticated state-machine-replication discipline of PBFT and locked-QC protocols [19, 21], specialized to independent name generations.

5.1 Adversary and authenticated state

For one namespace, the adversary controls at most f of $n_c = 3f + 1$ fixed validators. A Byzantine validator may equivocate, omit, delay, replay, or collude; the network may reorder, duplicate, drop, or partition messages. The adversary cannot forge Ed25519 signatures, find SHA-256 collisions, or break XChaCha20-Poly1305. Honest validators persist votes and locks before returning a signature. Owner or reader endpoint compromise, denial of service by more than f validators, and Sybil-resistant committee admission are outside the guarantee.

An owner-signed *NamespaceRoot* at `/namespace-roots/<NamespaceID>` binds the namespace, owner, exactly $3f + 1$ distinct Ed25519 validator keys, f, and format epoch. The shared CAS authority fixes the first root; v1 rejects every later committee change. This deliberately avoids a cross-committee safety claim without a reconfiguration certificate. Each vote signs its phase, name, generation, current and previous record hashes, committee epoch, and view under a distinct domain. A QC contains $2f + 1$ valid votes from distinct root members; duplicate signers do not count. The DHT transports certified records but does not choose among uncertified values.

5.2 Agreement protocol

Figure 2 shows the implemented path. Before voting, every honest validator checks root and owner binding, path-derived NameID, generation and predecessor, record/capability signatures, policy bounds, and the strict data-readiness gate. It then follows these rules:

1. In *prepare*, a validator signs at most one proposal per name, generation, phase, and view.



Fig. 2. Authenticated head-update path. The policy gate concerns immutable data availability; the three voting phases concern mutable-state agreement.

2. A prepare QC justifies *precommit*. Before returning its precommit vote, the validator durably locks the proposal hash. A restart or concurrent request cannot make it sign a conflicting hash for that generation.
3. A same-view precommit QC justifies *commit*. The commit node rechecks the certificate, data policy, root, and expected generation, then atomically performs CAS and marks the name certification-required.
4. Followers validate and monotonically persist the head, root, and QC. Completion requires q durable QC copies; compatibility update and rename paths thereafter reject the certified name.

5.3 Agreement and availability guarantees

Two quorums of $2f + 1$ among $3f + 1$ validators intersect in at least $f + 1$ members, including at least one honest validator. Suppose conflicting records for one name and generation both obtained commit QCs. Their precommit quorums intersect in an honest validator, which would have to persist locks on two hashes for the same generation. Journal serialization and durable non-equivocation forbid this contradiction. Thus both records cannot commit while the root and fault threshold hold. The CAS is an additional serialization check, not the source of the quorum proof.

Safety does not require synchrony. Progress requires at least q responsive validators and eventual delivery among them. The current coordinator retries the same locked proposal in a higher view. Its liveness profile therefore assumes a responsive coordinator; automatic leader election and general view change extend that profile to coordinator failure. A certified DHT read authenticates the returned head. Quorum reads with q authenticated receipts and write-back would additionally prevent a withholding network from returning a stale certified generation.

Seven has two separate roles. A seven-member committee is the minimum $3f + 1$ for $f = 2$, and five is its agreement threshold. Independently, the default data policy requests seven stored copies and currently advances the head only after all seven signed provider claims exist. That all-seven gate improves crash availability and makes publication wait for every selected provider. The separate five-of-seven validator quorum supplies Byzantine agreement; content hashes authenticate the bytes held by storage providers.

6 Search and Coordination

6.1 Search plane

The Prefix Hash Tree (PHT) and Bloom summaries occupy a dedicated `/pht/names/` keyspace. An entry represents one current, searchable logical name. Neither provider claims nor old generations enter this index. Updating a name replaces its search state; tombstoning or hiding it deletes the entry. Thus, for N searchable logical names, V versions per name, and R replicas, name-index cardinality is $O(N)$ by construction rather than $O(NVR)$. Total system metadata is not constant: manifests, retained versions, provider claims, and leases still consume space in their own planes.

Prefix and suffix queries fan out across configured shards. Each candidate is parsed into a NameID, resolved directly through the name DHT, and accepted only if its signed current head remains searchable and matches the query. Responses include shards attempted and completed, pending index repairs, and an explicit completeness bit. An index failure never rolls back a committed head; it records repair work and makes subsequent search incomplete until idempotent reconciliation succeeds. Exact lookup remains available without the index.

6.2 Algorithmic costs

Let n be overlay peers, α Kademlia parallelism, m current searchable names, p PHT nodes visited, c returned candidates, S object bytes, $u = \lceil S/B \rceil$ chunks, δ changed chunks, r copies, and n_c committee members. Kademlia shortens remaining XOR distance by routing prefix, so with populated buckets an exact NameID lookup takes expected $O(\log n)$ sequential rounds and $O(\alpha \log n)$ RPCs [15]. Name derivation is linear in path length and constant in versions and replicas.

A secondary search costs $O((p + c)\alpha \log n)$ overlay RPCs because each PHT node and verified candidate is a DHT lookup. Bloom summaries reduce expected p, not its worst case: an unselective suffix may visit $\Theta(m)$ entries. The name index is $\Theta(m)$; history and provider metadata still grow as $\Theta(mv)$ and $\Theta(ru)$ in their separate planes.

Building an update reads and authenticates $\Theta(S)$ bytes even if little changed. Same-epoch reuse reduces new ciphertext to $O(\delta B)$ and transmitted data to $O(r(\delta B + M))$ for manifest size M. Each agreement phase uses a coordinator star: at most n_c requests and q replies, hence $O(n_c)$ messages and signature operations rather than all-to-all traffic. The default committee is fixed size, so overlay discovery retains the $O(\log n)$ routing term while agreement work is constant in n.

6.3 Tuple space as coordination

Tarsus retains its native tuple space for storage offers, repair intents, transient work, and application queues. Stable objects use the direct name plane, while coordination uses associative matching. Exact-name queues serialize at a deterministic owner and preserve FIFO within that owner. Associative queries over multiple names are multiset matches and provide no global FIFO order. The TSH interface is an optional project-lineage compatibility adapter, independent of the mutable-name design.

The multiset is intentional: duplicate offers are independent consumable events that a key–value map would collapse, and templates select by known fields without a prearranged key. Executable work is represented as passive task types mapped to local allowlisted handlers. This preserves tuple coordination without adding mobile-code sandboxing and accounting to the storage trusted base.

7 Implementation

The prototype is implemented in Go over libp2p and its Kademlia-derived DHT. Kademlia itself is a replicated key/value lookup protocol, not an immutable filesystem [15]. Strict DHT validators cover derived keys, deterministic encodings, signatures, capabilities, leases, provider claims, namespace roots, and certified records; transition and CAS checks run at the mutation service. The new path never uses the legacy rule that any non-empty token is valid.

The raw `/put` and `/get` block API remains. Versioned routes create, resolve, update, delete, preflight, and search names; acquire or renew leases; persist BFT votes; perform certified CAS; and install certified heads on followers. All mutations carry signed expected-generation state.

The `tarsusctl object put|get|update|delete|search` client performs chunking, encryption, key wrapping, signing, staging, preflight, CAS, and verified reconstruction. Keeping private keys and plaintext processing at the client narrows the storage-node interface to ciphertext and signed public metadata.

The directory client adds `mkdir`, `link`, `unlink`, and `ls`; `GET /v1/directories/{id}` returns the verified directory head and its resolved current children. This is a protocol namespace rather than a graphical interface: applications can project it as folders while authorization and concurrency operate on the signed paths.

We define *network-scale fault-tolerant parallel computing* as addressed peers concurrently performing transfer, verification, search fanout, repair, and coordination under the stated failure mechanisms. The definition captures protocol parallelism and fault handling across the overlay; Sections VI and IX specify its asymptotic cost and partition behavior. ACAN and SMC record the project lineage.

Existing raw immutable operations and tuple-space tests remain in the regression suite. Multi-host scripts accept an explicit inventory containing bind and advertised addresses for three to five hosts; they do not assume container hairpinning or discover a public address. The scripts preserve node identities, health, neighbors, object operations, replica status, crash time, post-crash digest, and logs.

8 Evaluation

8.1 Experimental questions and setup

The evaluation tests whether decoupling pays off and whether authenticated, strict replication remains practical. We measure logical-name scaling, encrypted reuse and repair, directory mutation, authenticated threshold behavior, and policy cost relative to Bee. Experiments ran on an Intel Core i7-12700K desktop with 20 logical CPUs. Live

Table 1. Selected adversarial acceptance gates

Injected condition	Observed result
Concurrent siblings; raw update after certification	One CAS winner; downgrade rejected across API nodes
Four/duplicate/altered votes or a conflicting locked proposal	Certificate or equivocation rejected
Unauthorized state or a second committee root	Signature, capability, and root validation reject it
Overlapping exact/subtree leases or invalid directory child	Scope validation rejects the operation
Six provider claims or altered record/data/envelope	Publication withheld or cryptographic verification fails
File child advances with unchanged parent	Listing resolves the new child generation without rewriting ancestors

Tarsus cells used 10, 30, and 40 Docker peers; readiness passed before measurement. The 40-peer stress cell deliberately capped unprotected connections at two. Because all peers shared one bridge, the campaign measures protocol/process scaling and latency-class placement rather than independent failure domains.

Agreement, object, and repair cells were initialized once. Comparative results use three fresh 20-node deployments per system under one code and workload configuration. Bee uses an official five-node factory core plus 15 funded, unmodified Bee 2.8.1 containers.

8.2 Correctness and robustness gates

The complete Go test suite and `go vet` pass. Race-enabled tests pass for the name and control paths. A final 30-second signed-structure fuzz run also passes after 4,325,783 executions.

Table 1 compresses the main adversarial gates. Concurrent CAS and lease tests use separate service instances against shared authorities, not only goroutines sharing one mutex.

8.3 Authenticated agreement in a live overlay

Figure 3 reports the fresh 30-peer principal campaign. Seven persistent node identities formed the namespace committee, while the coordinator contacted only five: the execution therefore models two unavailable validators. Prepare, precommit, and commit phases took 135.59, 47.92, and 42.73 ms. Certified CAS followed by durable write-back to five members took 2.22 s end to end.

Four votes could not form a QC, corresponding to three unavailable committee members. The same live campaign rejected a signature-tampered vote, a conflicting proposal after a durable precommit lock, and a subsequent raw API update. Together these checks exercise omission, alteration, equivocation, and downgrade defenses at the configured threshold.

8.4 Overlay scaling and connection capacity

At 30 peers, observed neighbor counts ranged from 8 to 20 (mean 13.53), and DHT routing tables from 14 to 28 entries. CAS plus five write-backs took 2.22 s versus 2.12 s at 10 peers; strict 8-MiB publication rose from 0.88 to 4.09 s.

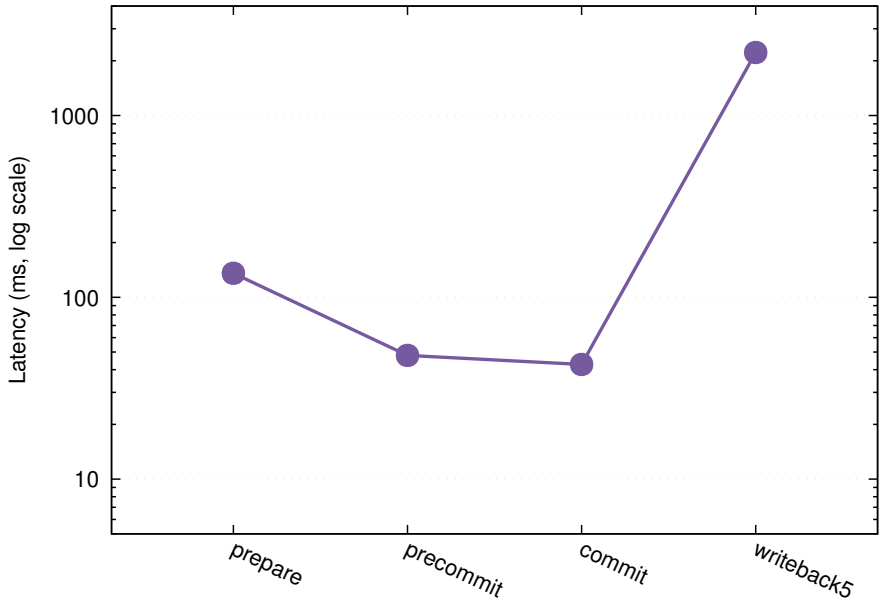


Fig. 3. Thirty-peer authenticated agreement. Five signatures suffice with two unavailable validators; four do not. Latency includes network service paths.

At 40 peers, the connection cap allowed only two unprotected links. Safety tests still passed, but CAS/write-back took 99.73 s, strict create took 83.36 s, and the provider target did not stabilize within 120 s. The 30-peer cell sustains the full protocol path; the capped 40-peer cell localizes the next scaling bottleneck to connection capacity rather than the name-index design.

8.5 Logical-name scaling

An in-memory campaign constructed and validated signed histories while keeping one current index entry per name. At 100,000 names, 1, 4, 16, 32, and 64 generations produced up to 6.4 million records in 7.19–471.50 s while the index remained at 100,000 entries. A two-million-name, one-generation cell completed in 141.10 s with two million index entries.

Record count and construction time are approximately linear in names times versions, whereas current-name index cardinality depends only on names. The 6.4-million-record cell peaked at 9.45 GiB resident and the two-million-name cell at 4.13 GiB. Across the measured range, the index remains proportional to logical names through two million names and 64 retained versions per name.

A separate exact-resolution microbenchmark populated 1,000 names and retained one, four, sixteen, 32, or 64 generations per name. Across five fixed-work runs, median current-head lookup was 7.05, 6.53, 5.73, 5.62, and 6.41 μs; the 64-generation range

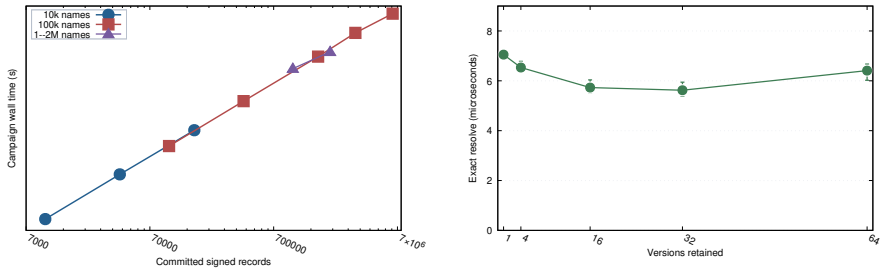


Fig. 4. Metadata scaling. Construction grows with signed-record count; measured exact-head lookup does not grow with retained versions.

was 6.02–6.67 μs. Lookup work did not grow with a 64-fold history increase in this local name-plane range; overlay latency adds the expected $O(\log n)$ routing term.

8.6 Directory mutation against Swarm

We isolate the directory metadata boundary: both systems retarget one existing leaf to a pre-staged immutable 1-KiB payload and publish current metadata, with encryption and foreground replication disabled. Tarsus uses 20 peers and single-owner CAS. Bee uses an official five-node factory core plus 15 unmodified Bee 2.8.1 containers (API 8.1.0, bee-js 12.3.1, redundancy zero), recursively loads a Mantaray, replaces one fork, calls `saveRecursively`, and publishes one signed sequence-feed update [12]. Excluding payload staging and recursive load from primary timing favors Bee.

We sweep binary depth 1–8 and depth-two fanout 4–32 (1,024 leaves), with five mutations in each of three fresh deployments. Figure 5 reports medians of deployment medians and their ranges. All changed leaves pass remote digest verification. All 135 Tarsus updates publish one 617–645-byte head and leave every ancestor at generation zero, confirming $O(1)$ ancestor rewrites. At depth eight (256 leaves), Tarsus takes 24.43 ms versus Bee's 1.56 s and one feed update; Bee issues 512 manifest uploads (122,816 bytes), deduplicated to 10 unique nodes (2,560 bytes). At 1,024 leaves, Tarsus and Bee take 300.46 ms and 3.52 s, and Bee issues 1,190 uploads (326,720 bytes).

For 32 disjoint changes, Tarsus publishes 32 independent heads concurrently in 77.80 ms without changing the root; Bee's favorable one-root application batch takes 286.21 ms. The result demonstrates the architectural advantage directly: independent child heads avoid recursive ancestor publication. Content deduplication keeps Bee's unique new state small, and a specialized persistent-trie client could improve on `saveRecursively` by caching unchanged subtrees.

8.7 High-assurance publication against Bee

Three fresh 20-process overlays upload high-entropy objects and verify remote SHA-256 digests. The four labeled contracts compare first-copy acknowledgment, public

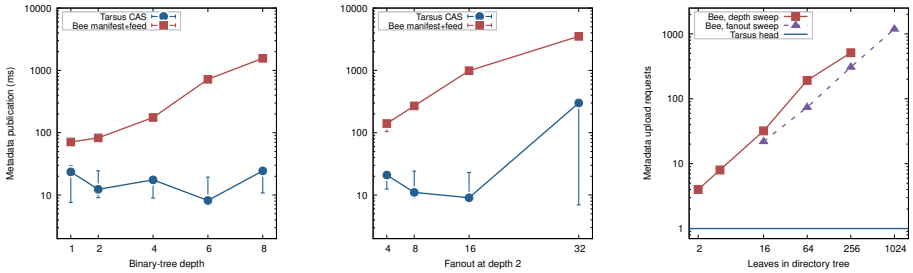


Fig. 5. Leaf-update metadata in three fresh 20-node deployments (five trials per shape). Points are medians of deployment medians; bars span them. The right panel counts Tarsus's one head request versus Bee Mantaray uploads.

publication, encryption, and the strongest tested paths. Tarsus's last row includes seven verified providers, independent checks by four non-proposer voters, three 5-of-7 phases, certified CAS, and five write-backs; Bee uses erasure level 4.

An initial Bee driver repeated one 32-byte hash and favored deduplication; those files are retained but excluded. The reported 48 Bee trials use independently sampled high-entropy bytes and pass length, digest, and stewardship checks.

Figure 6 reports 64-MiB medians of deployment medians. First-copy latency is 0.706 s for Tarsus and 7.977 s for Bee (11.29×). Strict-public Tarsus takes 4.724 s versus Bee direct r0's 63.914 s (13.53×). Strict-private Tarsus takes 4.924 s versus Bee encryption's 80.151 s (16.28×). Tarsus strict-private plus BFT takes 8.925 s versus Bee level 4's 192.514 s (21.57×). Every measured head remained hidden until its Tarsus preflight succeeded, and all remote reconstructions matched. Dividing 64 MiB by these end-to-end medians gives 90.62 versus 8.02 MiB/s for first copy, 13.55 versus 1.00 MiB/s for public, 13.00 versus 0.80 MiB/s for private, and 7.17 versus 0.33 MiB/s for the strongest tested paths.

The strongest pair prices distinct high-assurance contracts. Bee adds parity for reconstruction. Tarsus verifies seven complete providers and certifies mutable-head agreement through its five-of-seven validator quorum.

Bee wins remote-read latency: its public and encrypted reads take 14.81 and 10.54 ms after synchronization, versus 1.37 and 0.85 s for Tarsus raw and private paths. This identifies parallel retrieval and failed-provider avoidance as the highest-leverage Tarsus read-path optimizations.

Bee deferred acknowledgment takes 7.98 s, but completed remote retrieval follows 204.46 s later; this availability delay is not added to the bar.

All 12 Bee crash objects reconstructed with 15, 10, or 5 nodes remaining. Tarsus likewise resolved certified heads and reconstructed at each tier, including after the five original voters stopped.

Bee also executed postage, accounting, and settlement, which Tarsus lacks. Because the local wallets were test-funded and stamps did not expire, this campaign measures the

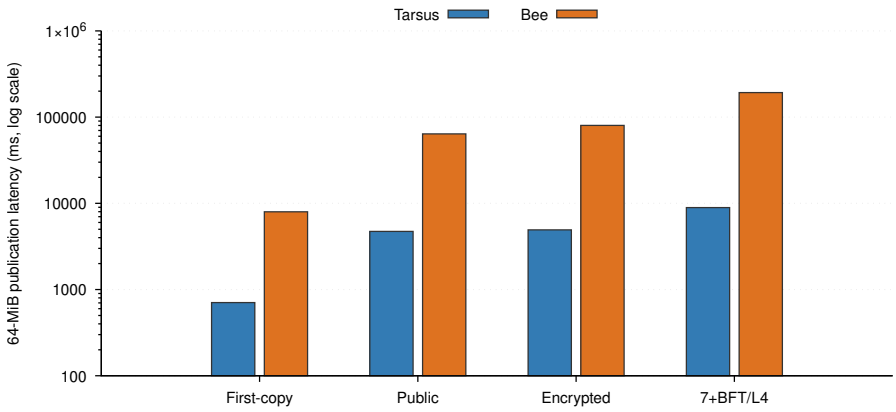


Fig. 6. Twenty-node 64-MiB high-entropy publication cost, three fresh deployments. Rows price the documented durability and agreement contracts. Lower is better.

protocol accounting path rather than long-term economic persistence among independent operators.

8.8 Encrypted update, retrieval, and repair

In the 30-peer deployment, strict publication placed a private 8-MiB object's eight chunks and manifest at seven providers in 4.09 s. An unchanged-content update took 1.05 s, reused all chunks, and emitted one new manifest.

An authorized peer reconstructed it in 0.15 s; a valid but unauthorized X25519 key lacked an envelope and was rejected.

After one provider stopped, the controller restored seven at 45 s; post-repair retrieval took 0.20 s and reproduced the digest.

8.9 Local cryptographic and data cost

Five-run private-object construction reaches 22.46, 390.07, and 449.12 MiB/s at 64 KiB, 4 MiB, and 64 MiB. These in-memory values include encryption, envelopes, manifest signing, and hashing but exclude network and disk.

One NameRecord sign-and-validate operation takes a median 48.36 μs and allocates 2,883 bytes. Validating a five-signature QC takes 156.04 μs and approximately 4,445 bytes. A sequential in-process three-phase five-of-seven protocol takes 3.39 ms, versus 2.22 s for live certified CAS and write-back; network and authority paths, not primitive signature verification, dominate that live operation.

9 Threat Model and Guarantee Boundaries

Peers may crash, omit messages, replay old records, return altered bytes, equivocate, or partition. Secondary indexes may be incomplete or stale. Under SHA-256 collision resistance and the assumed Ed25519, X25519, HKDF-SHA256, and XChaCha20-Poly1305

security, the prototype detects malformed or altered records, manifests, ciphertext chunks, envelopes, capabilities, leases, votes, QCs, and provider claims. Immutable keys authenticate provider bytes; private HMAC tags additionally bind plaintext to an epoch key without publishing a dictionary-testable digest.

For certified names, safety assumes one immutable, owner-authenticated committee root, at most f Byzantine members, durable honest-validator journals, and unbroken cryptography. Section 5 then gives the quorum-intersection argument. If more than f members equivocate, the owner installs malicious validators initially, journal durability is lost, or keys are compromised, conflicting certificates can become possible. V1 rejects committee replacement until a reconfiguration certificate can authenticate the membership transition. It also lacks automatic leader/view change and quorum-read receipts: a returned certificate is authentic, but a withholding network can make a read stale or stop progress. The compatibility CAS path has a weaker crash/partition model and cannot update a name once certified mode is set.

Strict publication makes an under-replicated new head invisible, but signed provider claims prove possession only at verification time. Seven stored copies are an availability target, not an agreement quorum, and an all-seven gate lets one withholding provider delay publication. A malicious provider can later omit a block; repair and hash verification recover only if another valid copy remains. The evaluated providers share one host and bridge, while RTT classes identify latency rather than geography, power, rack, jurisdiction, or administrative independence.

Client-side encryption excludes storage providers and readers without an envelope, assuming uncompromised owner/reader endpoints and keys. Public searchable paths, sizes, access patterns, provider addresses, and timing still leak. Private keyed tags reduce offline confirmation of guessed content, but chunk lengths and same-epoch reuse can reveal equality patterns. Reader revocation governs future epochs; it cannot retract a key or plaintext already obtained.

A tombstone removes the current namespace reference and triggers best-effort collection. Destroying every remaining envelope may provide cryptographic erasure to parties that never learned the epoch key, but no protocol can force a third party to delete bytes, keys, screenshots, or plaintext already copied. Search reports incomplete fanout instead of claiming complete enumeration. Byte-range concurrency remains application-controlled, and associative tuple matching remains unordered across names; only exact owner-local queues retain FIFO.

10 Related Work

IPFS, IPNS, and IPFS Cluster. IPFS provides immutable Merkle-DAG objects; IPNS adds signed versioned pointers resolved through the DHT, and IPFS Cluster adds allocation and repair [1–5]. Tarsus binds per-object policy to the signed path head and requires provider evidence before CAS.

Swarm. Swarm combines signed feeds, immutable manifests, encrypted references, ACT access metadata, and configurable redundancy [7, 8, **?**,9, 11, 14]. Tarsus adds exact path-derived CAS records, current-name search, an owner-selected provider predicate,

and five-of-seven head certification. Because every Tarsus child has an independent NameID, mutation avoids Swarm's recursive manifest publication [12]. Bee retains advantages in retrieval, incentives, tooling, compact tries, and erasure coding [13]; the evaluation reports both systems' measured strengths.

DHTs and secondary search. Kademlia supplies XOR key/value routing [15]; PHT and Bloom filters supply secondary search [23, 24]. Tarsus re-resolves every candidate against its signed current head.

Byzantine agreement and concurrent state. PBFT and Byzantine quorum systems establish the $3f + 1$ intersection basis [19, 20]; HotStuff demonstrates responsive QC-based agreement [21]. Tarsus specializes durable locks and fixed membership to independent name generations. Applications may store a CRDT payload when mergeable state is preferable to head CAS [22].

Tuple coordination. Linda's associative tuple spaces provide the model for Tarsus offers and work [25, 26]; stable names use direct DHT keys.

11 Conclusion

Tarsus turns immutable encrypted blocks into stable mutable named objects by making one signed generation record the authority for both the current manifest and per-object policy. Exact names, search entries, provider claims, coordination tuples, and bytes remain separate planes. Expected-generation CAS serializes compatibility updates; the hardened profile adds a fixed $3f + 1$ committee, durable locks, $2f + 1$ certificates, follower write-back, and downgrade prevention. Strict publication hides under-replicated heads, while client-side epochs provide authenticated encryption and changed-chunk reuse.

Five-of-seven agreement survives two unavailable validators and rejects the four-vote, altered-vote, locked-fork, and downgrade cases. Encrypted updates reuse unchanged chunks, repair restores the provider target, and current-name indexing remains independent of retained versions over the tested range. The evaluation shows that this decoupling pays at both metadata and data scale: updates avoid directory-ancestor rewrites, exact lookup avoids history scans, and strict encrypted publication remains fast even with seven providers and authenticated agreement. Tarsus thereby establishes a practical mutable-name architecture in which policy and content evolve independently without expanding the current-name index.

Acknowledgments. This research is funded in part by the Temple University Innovation Flight Fund #276206 and NSF Chameleon compute resource grant #CHI-251439.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. J. Benet, "IPFS—Content Addressed, Versioned, P2P File System," 2014, arXiv:1407.3561.
2. Protocol Labs, "Immutability," IPFS Documentation. [Online]. Available: https://docs.ipfs.tech/concepts/immutability/
3. Protocol Labs, "IPNS (InterPlanetary Name System)," IPFS Documentation. [Online]. Available: https://docs.ipfs.tech/concepts/ipns/
4. Protocol Labs, "Distributed Hash Tables (DHT)," IPFS Documentation. [Online]. Available: https://docs.ipfs.tech/concepts/dht/
5. IPFS Cluster, "Adding and pinning," IPFS Cluster Documentation. [Online]. Available: https://ipfscluster.io/documentation/guides/pinning/
6. Protocol Labs, "Filecoin: A decentralized storage network," 2017. [Online]. Available: https://filecoin.io/filecoin.pdf
7. Ethereum Swarm, "Feeds," Swarm Documentation. [Online]. Available: https://docs.ethswarm.org/docs/develop/tools-and-features/feeds/
8. Ethereum Swarm, "Manifests," Swarm Documentation. [Online]. Available: https://docs.ethswarm.org/docs/develop/tools-and-features/manifests/
9. Ethereum Swarm, "Access Control," Swarm Documentation. [Online]. Available: https://docs.ethswarm.org/docs/concepts/access-control/
10. Ethereum Swarm, "Store with Encryption," Swarm Documentation. [Online]. Available: https://docs.ethswarm.org/docs/develop/tools-and-features/store-with-encryption/
11. Ethereum Swarm, "Distributed Immutable Store of Chunks," Swarm Documentation. [Online]. Available: https://docs.ethswarm.org/docs/concepts/DISC/
12. Ethereum Swarm, "Manage Files," Swarm Documentation. [Online]. Available: https://docs.ethswarm.org/docs/develop/files/
13. Ethereum Swarm, "Bee API," Swarm Documentation. [Online]. Available: https://docs.ethswarm.org/api/
14. Ethereum Swarm, "Erasure Coding," Swarm Documentation. [Online]. Available: https://docs.ethswarm.org/docs/concepts/DISC/erasure-coding/
15. P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in *Proc. IPTPS*, 2002.
16. C. Bormann and P. Hoffman, "Concise Binary Object Representation (CBOR)," RFC 8949, Dec. 2020.
17. S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, Jan. 2017.
18. A. Langley, M. Hamburg, and S. Turner, "Elliptic Curves for Security," RFC 7748, Jan. 2016.
19. M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. OSDI*, 1999, pp. 173–186.
20. D. Malkhi and M. Reiter, "Byzantine quorum systems," *Distributed Computing*, vol. 11, no. 4, pp. 203–213, 1998.
21. M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "HotStuff: BFT consensus with linearity and responsiveness," in *Proc. PODC*, 2019, pp. 347–356.
22. M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Proc. SSS*, 2011, pp. 386–400.
23. S. Ramabhadran, S. Ratnasamy, J. M. Hellerstein, and S. Shenker, "Prefix Hash Tree: An indexing data structure over distributed hash tables," in *Proc. PODC*, 2004.
24. B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.
25. D. Gelernter, "Generative communication in Linda," *ACM Trans. Program. Lang. Syst.*, vol. 7, no. 1, pp. 80–112, 1985.
26. N. Carriero and D. Gelernter, "Linda in context," *Commun. ACM*, vol. 32, no. 4, pp. 444–458, 1989.
27. Author, F.: Article title. Journal **2**(5), 99–110 (2016)
28. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) CONFERENCE 2016, LNCS, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). https://doi.org/10.10007/1234567890
29. Author, F., Author, S., Author, T.: Book title. 2nd edn. Publisher, Location (1999)
30. Author, A.-B.: Contribution title. In: 9th International Proceedings on Proceedings, pp. 1–2. Publisher, Location (2010)
31. LNCS Homepage, http://www.springer.com/lncs, last accessed 2023/10/25